Datum
YOUR DATA. DECODED

PRODUCT REQUIREMENTS DOCUMENT

Weekly Reports / Presentation Viewer

Bringing the weekly client presentation into Datum: brand-scoped, permission-controlled, and viewable without ever exposing the original file.

Version	Date	Audience	Status	Confidentiality
v0.1 (Draft)	19 October 2026	Product, Engineering, Design, Management	For review	Internal

Document Control

Version history

Version	Date	Change	Author
0.1	19 October 2026	First draft, based on repository review and product-owner decisions	Aryan Arora (with Claude Code)

Reviewers

Role	Name	Sign-off
Product owner		
Engineering lead		
Design		
Security reviewer		

Conventions

How to read this document: **Confirmed** = verified in the repository, **Decision** = agreed with the product owner, **Assumption** = our choice, not yet confirmed (collected in section 29), plus **Recommendation** (engineering advice with its reason) and **Open question** (needs a decision). Requirement IDs (FR, VW, SEC, AC) are stable so tests and tickets can reference them.

Contents

1	Executive Summary	4
2	Problem Statement	4
3	Goals	5
4	Non-Goals	5
5	Personas and Roles	5
6	User Stories	6
7	Feature Overview	6
8	Detailed Functional Requirements	7
9	Brand Scoping	8
10	Permission and Authorization Model	9
11	Internal User Flow	11
12	Client User Flow	12
13	Presentation Viewer Requirements	14
14	Upload and Processing Flow	14
15	Report Lifecycle	15
16	Versioning	16
17	Data Model	16
18	API Requirements	19
19	Storage Architecture	20
20	Security Requirements	20
21	Brand Isolation Test Matrix	21
22	Performance Requirements	22
23	Error Handling	22
24	Audit Logging	23
25	Watermarking	24
26	Canva (Future Extension)	24
27	V1 vs V2 Scope	24
28	Acceptance Criteria	26
29	Assumptions and Open Questions	27
30	Implementation Considerations	28
31	Future Enhancements	29

1. Executive Summary

Every week the Product Team presents a status and progress deck to each client brand in a meeting. The deck lives in PowerPoint or Canva, is shared outside Datum, and is hard to control: old versions circulate, the wrong brand's deck can be sent, and Datum has no record of what a client was shown.

Weekly Reports brings that workflow into Datum. An internal author or admin uploads a PPT/PPTX for the *currently selected brand*, previews the converted slides, and publishes it. Client users who hold the view permission see the latest published report for their own brand, and nothing else, in an in-app presentation viewer with slide navigation, thumbnails and fullscreen. The original file is never sent to the client browser and Datum offers no download, share, export or print function.

What this document decides

Topic	Direction for V1
Who uploads	Elevated roles only (author, admin, super_admin). Not grantable to clients.
Who views	Any user holding the new per-brand permission `weekly_reports_panel`, plus elevated roles.
Brand scope	Taken from Datum's current brand and re-verified by the backend from the gateway-signed identity. Never trusted from the request body.
Processing	New `weekly-reports-service` converts PPT/PPTX to per-slide WebP images on the server.
Storage	New dedicated private S3 bucket. No public URLs; slides are served only through the authorized API.
Versions	Never overwritten. Clients see only the latest published version.
Watermark	In V1: dynamic per-viewer watermark burned into slide images. It deters casual sharing; it does not stop screenshots.
Canva	Not in V1. Future import flow, always ending in the Datum viewer.

This PRD is written against the actual Datum repository (frontend, API gateway, auth, analytics and supporting services). Statements verified in the code are marked **Confirmed**; choices made with the product owner are **Decision**; things we assumed are **Assumption** and are collected in section 29.

2. Problem Statement

- **Fragmented delivery.** Decks are shown from personal PowerPoint or Canva sessions, so clients cannot revisit the latest report on their own and Datum is not the single place for brand performance.
- **No control over versions.** A corrected deck may be sent after an error; the earlier one is still in circulation.
- **Risk of cross-brand exposure.** Files are handled manually per brand with no system enforcing which brand sees which deck.
- **No audit trail.** Datum cannot answer who uploaded or published a given report, or when.
- **Original files travel.** Sending the PPTX or a Canva link hands the client an editable, forwardable source file.

3. Goals

#	Goal	How we know
G1	Deliver the weekly deck to clients inside Datum, brand-scoped.	Client opens latest report without leaving Datum.
G2	Enforce brand and permission rules in the backend, not only the UI.	Brand-isolation test matrix (section 21) passes.
G3	Keep the original file private; give clients no download, share, export or print path.	No endpoint or UI exposes the source; security tests pass.
G4	Make versioning safe: corrections never destroy history; clients see one live report.	Version and publish acceptance criteria pass.
G5	Fast, presentation-like viewing.	Proposed targets in section 22 are met.
G6	Record who did what.	Audit events exist for upload, publish, unpublish, delete.

4. Non-Goals

- Editing or authoring slides inside Datum.
- Presenting live to a meeting, speaker notes, slide animations, embedded video or audio playback.
- Preventing screenshots or screen recording (technically impossible in a browser; see section 20).
- Client access to historical reports, search across reports, comments or annotations.
- Canva URL import, PDF upload, email delivery of reports and per-client analytics (see section 27).
- Making any change to existing analytics, alerts or pipeline behavior.

5. Personas and Roles

Persona	Datum role today	Weekly Reports capability
Product Team member (uploads and presents)	`author` or `super_admin` (the gateway also treats `admin` as elevated)	View + manage: upload, preview, publish, unpublish, delete, see version history, for the currently selected brand. Elevated roles can switch to any brand.
Admin (access control)	Elevated role	Assigns `weekly_reports_panel` to client users per brand in Access Control.
Client brand user	`viewer` or `brand_user` with a brand membership	View only, if granted `weekly_reports_panel` for that brand: latest published report, viewer controls. No other control.
Client without the permission	Same roles, permission absent	No menu item; direct URL redirects away; API returns 403.

> **Confirmed in repo:** Datum has no separate 'internal vs client' flag. Roles are the only distinction: author, admin and super_admin are elevated and get global brand access with an implicit `all` permission; viewer and brand_user need a brand membership with explicit permissions (`api-gateway/gateway/lua/auth.lua` , `api-gateway/src/services/rbac.service.js`).

6. User Stories

ID	As a...	I want to...	So that...
US-1	Product user	upload a PPT/PPTX while viewing a brand	it is automatically tied to that brand
US-2	Product user	preview the converted slides before publishing	I catch conversion problems before a client sees them
US-3	Product user	publish, unpublish or replace a report	clients always see the right deck
US-4	Product user	upload a corrected version without losing the old one	history is preserved and clients see only the fix
US-5	Product user	delete a mistaken upload	it does not clutter version history
US-6	Client user	open the latest weekly report in Datum	I can review progress without a file or a meeting link
US-7	Client user	move through slides with buttons, thumbnails and keyboard, and go fullscreen	it feels like a presentation
US-8	Admin	grant or revoke the view permission per client and brand	only the right people see reports
US-9	Security owner	know the original file and other brands' reports are unreachable	client confidentiality holds

7. Feature Overview

Weekly Reports is a single new panel (`/weekly-reports`) that adapts to the signed-in user: management tools for elevated users, a read-only latest-report card and viewer for clients. A new backend service owns upload, conversion, storage and authorized slide delivery.

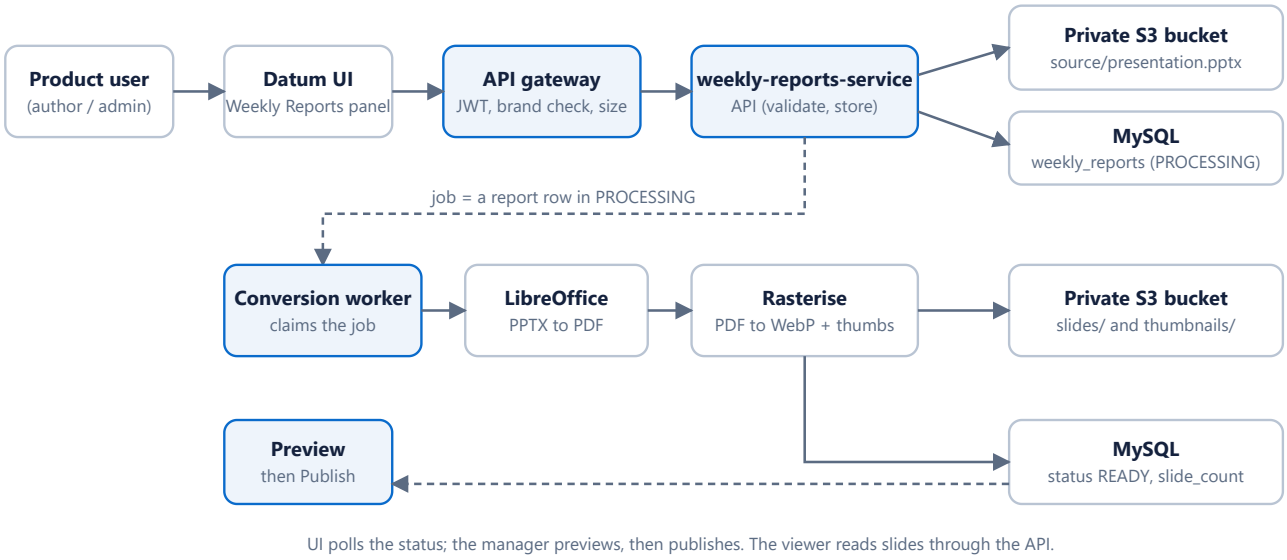


Figure 1. Upload and processing path (internal user).

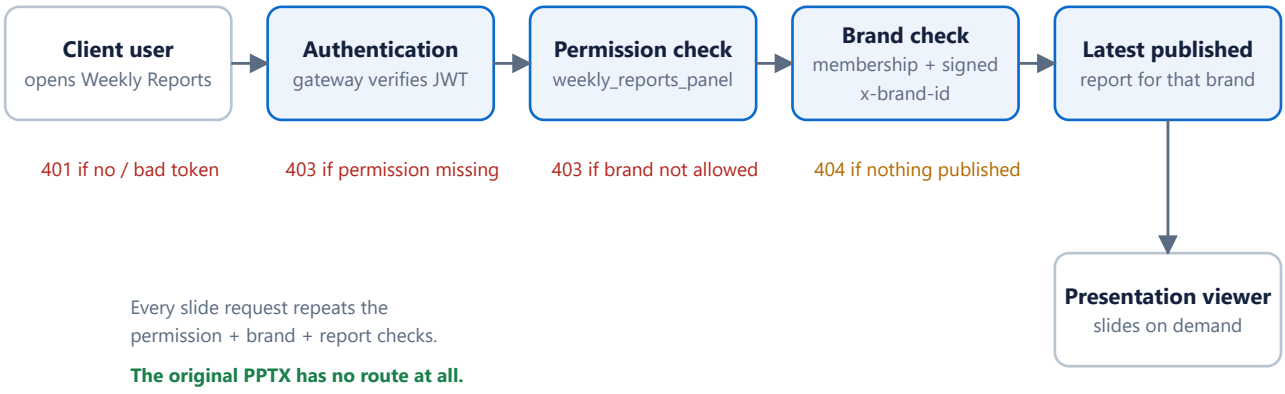


Figure 2. Client viewing path.

> **Decision:** Backend lives in a **new** `weekly-reports-service` container (API plus conversion worker) on the existing EC2 Docker Compose, behind the existing API gateway. Heavy document conversion stays out of the analytics container.

8. Detailed Functional Requirements

Upload and management (elevated roles)

ID	Requirement	Priority
FR-1	The panel shows an upload area for the **current brand**. There is no second brand selector.	Must
FR-2	Accept `.pptx` and `.ppt` only (content-checked, not just by extension), up to the maximum size (section 29, default 50 MB).	Must
FR-3	Collect a report title and a reporting week (free text label, for example 'Week 42').	Must

ID	Requirement	Priority
FR-4	On upload, create a report in PROCESSING and show progress; auto-refresh status until READY or FAILED.	Must
FR-5	Preview a READY report in the same viewer clients use, before publishing.	Must
FR-6	Publish makes a READY or UNPUBLISHED report the client-visible report for the brand; any previously published report for that brand becomes UNPUBLISHED automatically.	Must
FR-7	Unpublish hides the report from clients immediately.	Must
FR-8	Uploading for a brand and reporting week that already has a report creates the next version (v2, v3…). Nothing is overwritten.	Must
FR-9	Show version history for the brand to elevated users only.	Must
FR-10	Delete a report that is not currently PUBLISHED (soft delete; section 17).	Must
FR-11	Show failure reason on FAILED reports and let the user upload a replacement.	Must

Client viewing

ID	Requirement	Priority
FR-20	Panel appears only for users with `weekly_reports_panel` for the active brand (or elevated role).	Must
FR-21	Show one card: title, reporting week, published date, and a **View Report** button. The server decides which report this is.	Must
FR-22	If nothing is published, show an empty state; never an error and never another brand's report.	Must
FR-23	Open the presentation viewer (section 13).	Must
FR-24	No download, share, export, print, open-original, Canva-source, upload, delete, publish, unpublish or replace control or endpoint is available to clients.	Must
FR-25	Dynamic watermark on every slide served to a client (and in internal preview).	Must (V1)

Cross-cutting

ID	Requirement	Priority
FR-30	Every API call is authenticated, permission-checked and brand-checked in the backend.	Must
FR-31	Write an audit event for upload, version creation, processing result, publish, unpublish, delete and denied cross-brand attempts.	Must
FR-32	Failed or unpublished reports are never returned to a client user.	Must

9. Brand Scoping

A report belongs to exactly one brand, set by the server at upload time and never changed.

- **Frontend:** the panel reads the active brand from Datum's global brand state and sends it as the `brand_key` query parameter, like every other Datum API call. Switching brand reloads the panel for that brand.

- **Gateway:** verifies the user's membership for that brand and forwards a signed `x-brand-id` header.
- **Service:** uses only the signed header as the brand. A `brand_id` / `brand_key` in the body or URL path that disagrees is rejected (400); it is never used to choose a brand.
- **Every lookup** includes the brand: `WHERE id = ? AND brand_key = ?`. A report ID alone is never enough.

Confirmed in repo: The frontend stores the active brand in `brandSlice` (localStorage key `author_active_brand_v1`) and sends it as `brand_key` in the query string. The gateway resolves brand only from the query or headers; a body-only brand silently falls back to the caller's primary brand (`client/dashboard/src/lib/api.js`, `auth.lua`). The new endpoints must therefore read the brand from the signed header and the client must put `brand_key` in the query.

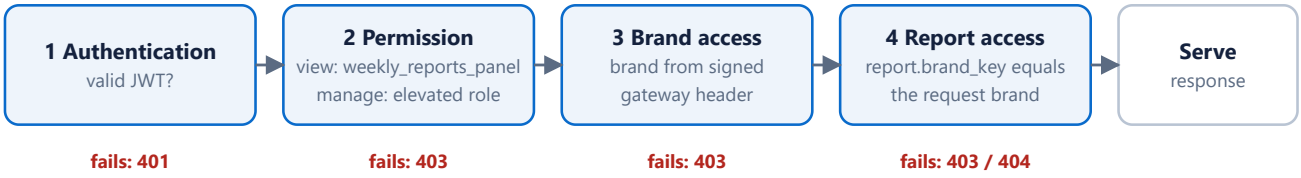
Confirmed in repo: For non-elevated users the gateway returns 403 'Access denied to this brand' when the brand is not in their memberships (`auth.lua`). Elevated roles pass for every brand.

10. Permission and Authorization Model

Datum stores permissions as flat snake_case strings on each user's per-brand membership. Panels use the `*_panel` suffix and sub-scopes use `name:sub`.

Capability	Who	How it is represented
View (see panel, open latest published report, use viewer)	Elevated roles; any user granted it for that brand	New permission string `weekly_reports_panel`, added to `ALL_PERMISSIONS` and the Access Control catalog.
Manage (upload, preview, publish, unpublish, version, delete, history)	Elevated roles only	Enforced as an elevated-role check in the service (same idea as `requireTrustedAuthor`). Not grantable to clients in V1.

Decision: Authors/admins have both capabilities automatically. Clients get **one** grantable permission that only allows viewing. Naming follows the repo convention (`weekly_reports_panel`) instead of the dotted `weekly_reports.view` in the original brief. If manage ever needs to be delegated, a reserved scope `weekly_reports_panel:manage` follows the existing colon convention.



All four checks run in the service on every request. The browser's own state (selected brand, hidden menu items) is never trusted.

Figure 3. Authorization chain applied to every request.

Enforcement points

- **Frontend (convenience only):** hide menu item, redirect guard, hide manage controls. Not a security boundary.
- **Gateway:** JWT verification and brand membership.

- **Service (authoritative):** signature check on the gateway headers, permission check, brand match, report-state check.

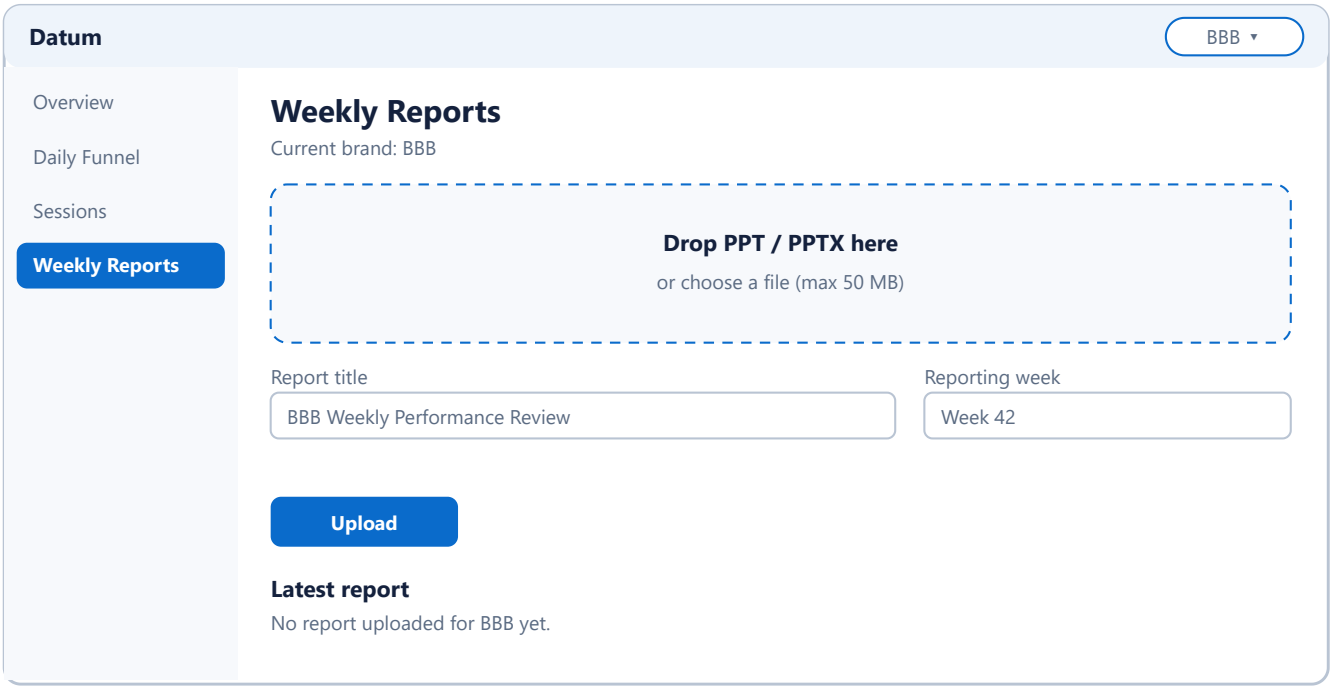
> **Confirmed in repo:** Gateway verifies the RS256 JWT and injects `x-user-id`, `x-brand-id`, `x-role`, `x-permissions` and an HMAC signature (`x-gw-sig`); services never see the JWT. Services use middleware such as `requirePermission(p)` followed by brand handling (`analytics/shared/middleware/identityEdge.js`). 401 is `{error:"Unauthorized"}`, 403 is `{error:"Forbidden"}`.

> **Confirmed in repo:** A new permission string must be added to `api-gateway/src/constants/permissions.js` (`ALL_PERMISSIONS` is the mongoose enum for memberships and custom roles) and to `PERMISSION_OPTIONS` in `client/dashboard/src/components/AccessControlCard.jsx`; otherwise it cannot be saved or assigned.

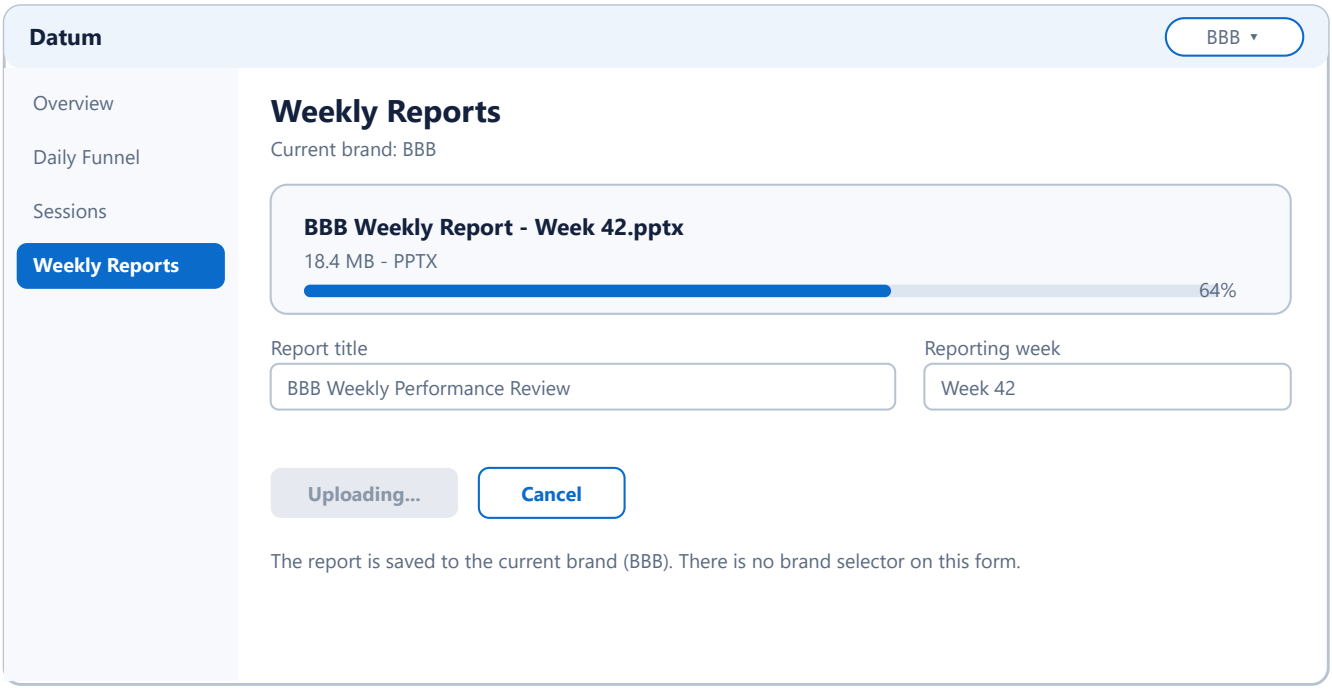
> **Recommendation:** Fail closed. In analytics, the signature check passes when `GATEWAY_SHARED_SECRET` is unset. The new service must instead refuse to start (or reject every request) without the secret, as `daily-insights-service` already does. Reason: a misconfigured environment must not silently disable authentication on a confidential-document service.

11. Internal User Flow

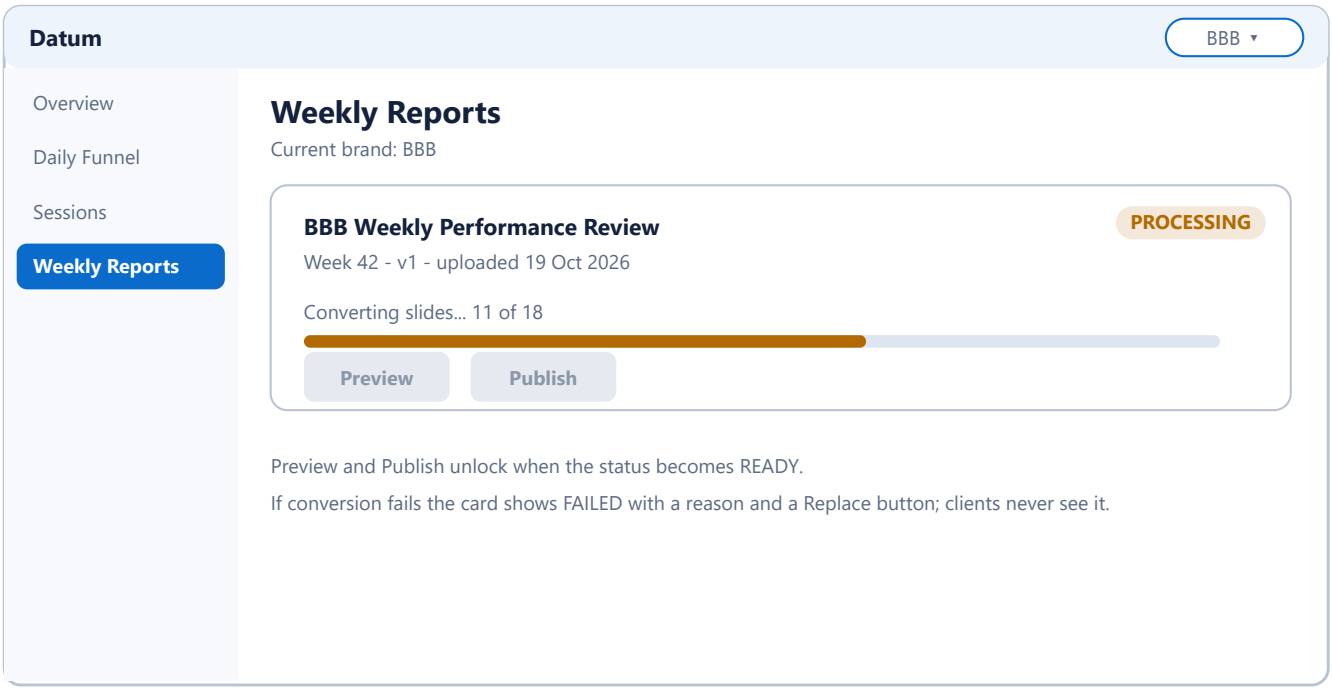
1. Open **Weekly Reports** with the target brand selected (for example BBB).
2. Choose or drop a PPT/PPTX, enter title and reporting week, click **Upload**.
3. The report appears as PROCESSING with progress; the page updates itself.
4. At READY, click **Preview** and check every slide (fonts, images, layout).
5. Click **Publish**. The client sees it on next load. Any earlier published report is unpublished.
6. If something is wrong: **Unpublish**, or **Upload new version** for the same week and publish v2.



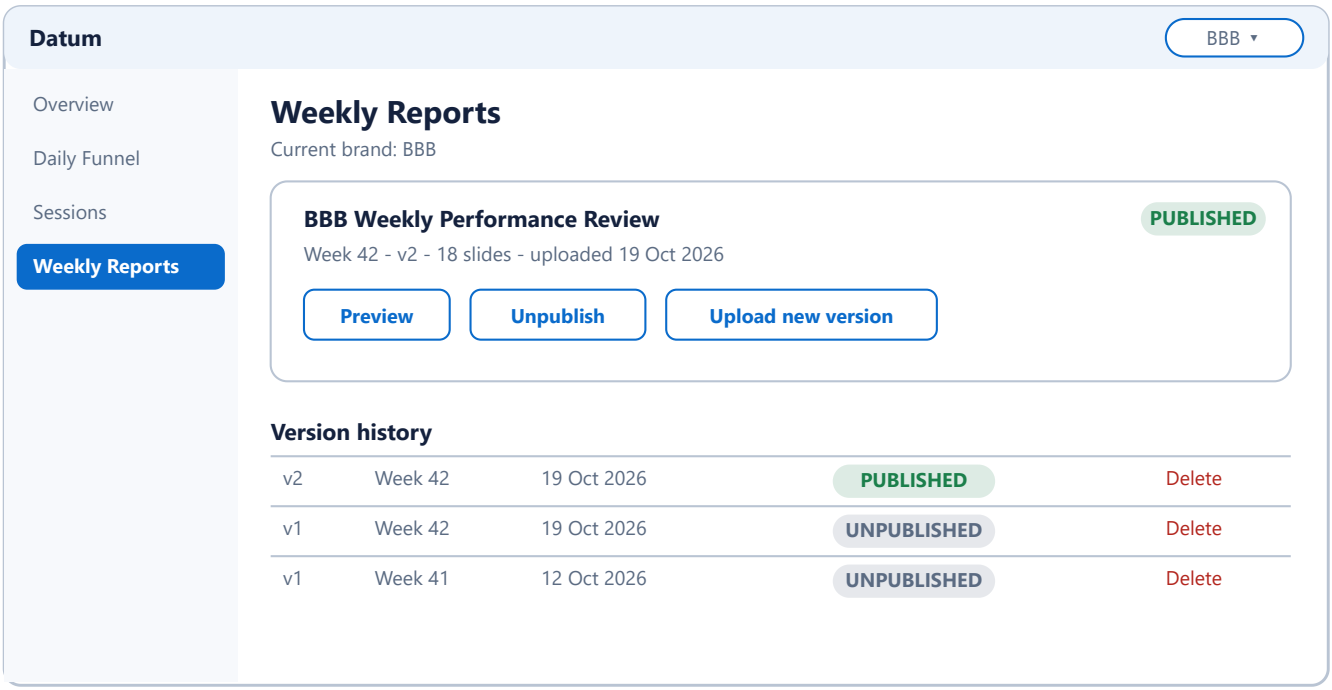
Wireframe 1 - Internal panel (author / admin), nothing uploaded yet



Wireframe 2 - Upload in progress (validation errors show under the file box)



Wireframe 3 - Processing state



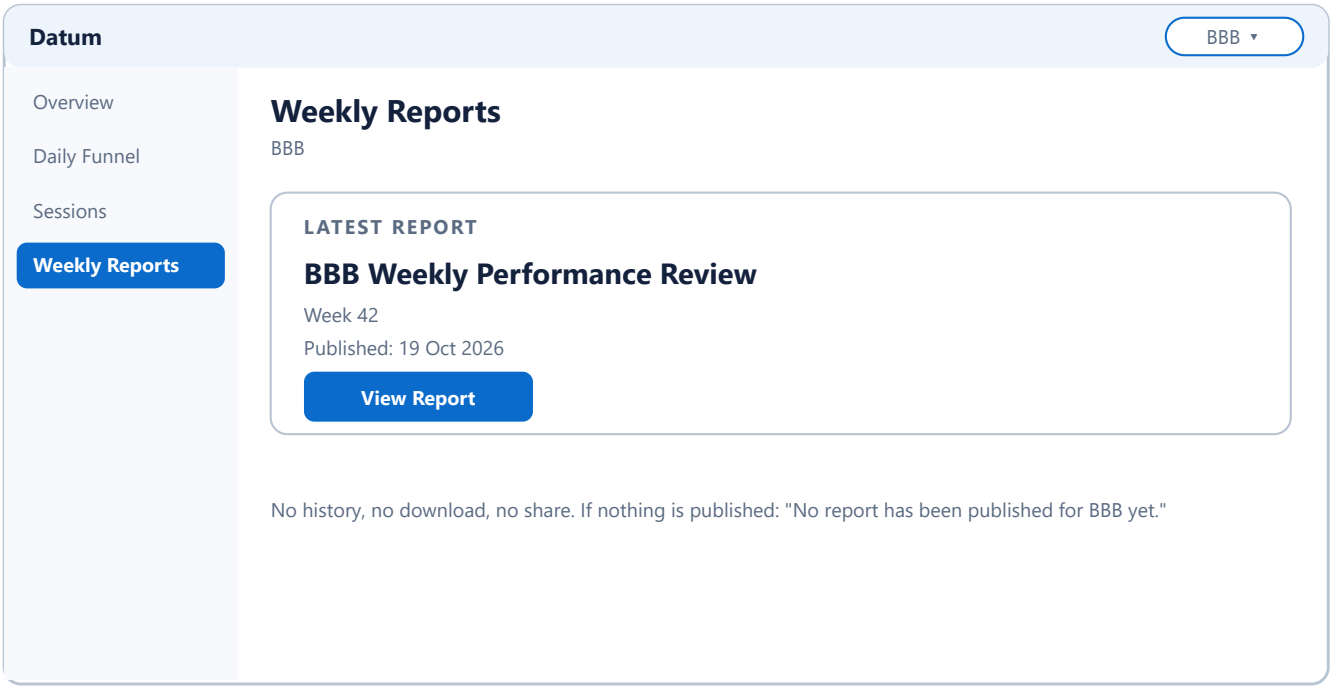
Wireframe 4 - Published report with version history (internal only)

Decision: One panel serves both audiences. Management controls render only for elevated roles; the backend independently rejects manage calls from anyone else.

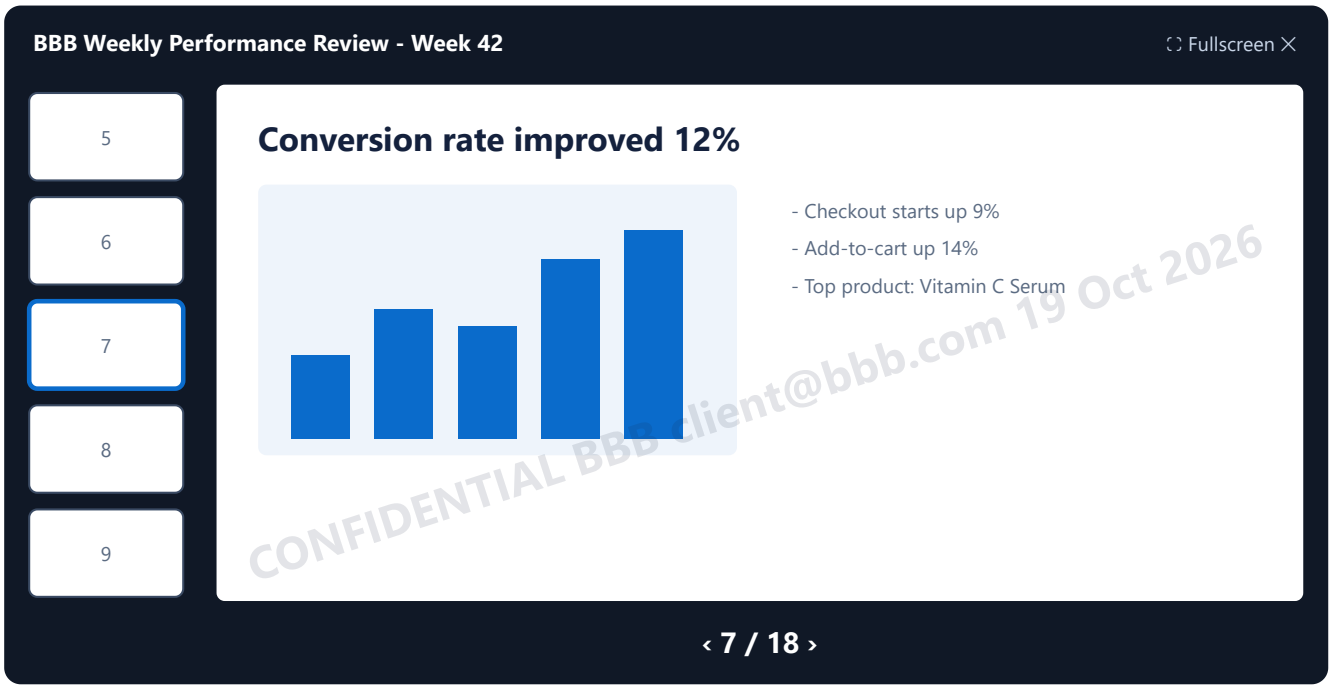
12. Client User Flow

1. Sign in. If the user holds `weekly_reports_panel` for the brand, **Weekly Reports** appears in the sidebar.
2. The panel shows the latest published report card.
3. Click **View Report** to open the viewer on slide 1.

4. Navigate, go fullscreen, close. There is nothing else to do.



Wireframe 5 - Client panel (view-only permission)



Wireframe 6 - Client presentation viewer (thumbnails, slide, controls, dynamic watermark)

A client cannot choose a report ID, browse older weeks, or see whether other reports exist. If a user belongs to several brands, the active brand selector decides which brand's report is shown.

13. Presentation Viewer Requirements

ID	Requirement
VW-1	Previous / next buttons, current slide number and total ('7 / 18').
VW-2	Thumbnail rail; clicking a thumbnail jumps to that slide. Thumbnails are small images loaded lazily.
VW-3	Fullscreen button using the browser Fullscreen API; `Esc` exits.
VW-4	Keyboard: `←` / `→`, `PgUp` / `PgDn`, `Home` / `End`, `F` for fullscreen. Touch: swipe left/right.
VW-5	Slide keeps its aspect ratio (16:9 and 4:3) and fits the container; responsive down to tablet width, usable on phone.
VW-6	Loads slide 1 first, prefetches the next two slides and the previous one, lazy-loads thumbnails. Never downloads the whole deck up front.
VW-7	Per-slide loading skeleton and a per-slide error state with Retry. A failed slide does not break the viewer.
VW-8	Light and dark theme support, consistent with the app (Tailwind `dark` class).
VW-9	No download, share, print, open-original or right-click-save controls. The viewer disables image dragging and the context menu on slides, and hides itself in print styles. These are deterrents, not security (section 20).
VW-10	Optional: zoom (V1.1).

Confirmed in repo: The frontend has no existing document or image viewer, no fullscreen code and no file-upload UI, so the viewer and uploader are new components (`client/dashboard/src` contains none of `type="file"`, `FileReader`, `requestFullscreen`). Reusable pieces: shadcn `dialog`, `tabs`, `skeleton`, `react-toastify`, lucide icons.

Recommendation: Fetch slide images with `fetch()` and the existing Bearer header, then show them through `URL.createObjectURL`. Reason: Datum authenticates with a Bearer token from localStorage, which a plain `<img src>` cannot send, and we do not want tokens in URLs. Revoke object URLs when slides leave the window.

14. Upload and Processing Flow

1. **Receive.** Gateway accepts the multipart upload (its body limit must be raised for this route), forwards to the service.
2. **Validate.** Elevated role, brand from signed header, size, extension and file signature (OOXML zip with `[Content_Types].xml` or legacy OLE), reject macro-enabled `.pptm`, guard against zip bombs.
3. **Store original** privately under the brand and report ID; insert the report row as PROCESSING; write the UPLOADED audit event; return 202 with the report ID.

4. **Convert (worker).** Claim the job, run LibreOffice headless (isolated profile, no network, hard timeout) to produce a PDF, rasterize each page, write full-size and thumbnail WebP images, count slides.
5. **Finish.** Insert slide rows, set READY with `slide_count`; on any failure set FAILED with a machine code and an internal message.

Processing details

Item	Recommendation (proposed)
Slide image	WebP, 1920 px wide, quality about 80, aspect ratio preserved.
Thumbnail	WebP, 320 px wide.
Max slides	100 (reject larger decks with a clear error).
Timeout	5 minutes per deck, then FAILED (`PROCESSING_TIMEOUT`).
Concurrency	1 to 2 conversions at a time; the rest wait.
Recovery	On service start, any report stuck in PROCESSING beyond the timeout is marked FAILED.

Confirmed in repo: There is no job queue library (no BullMQ or similar), and no LibreOffice, ImageMagick, poppler or sharp in any image or `package.json`. SQS is not used. Kafka exists but carries intent events and is operated separately.

Recommendation: Use a database-backed job claim (a row in PROCESSING claimed with `SELECT ... FOR UPDATE SKIP LOCKED`) instead of adding a queue product. Reason: one low-volume job type per week per brand does not justify new infrastructure. This requires MySQL 8; confirm the central database version (section 29).

Assumption: Conversion fidelity: LibreOffice renders most PowerPoint decks well but may substitute fonts and will not play animations or embedded video. Canva exports with custom fonts are the main risk. The mandatory preview-before-publish step (FR-5) is the mitigation, and the worker image should include the brand fonts used in decks.

15. Report Lifecycle

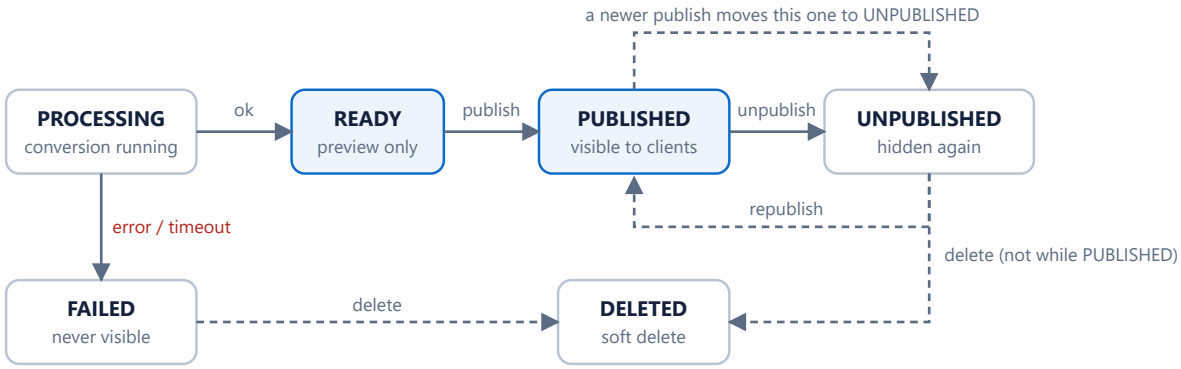


Figure 4. Report states.

State	Meaning	Client-visible
PROCESSING	Original stored, slides being generated.	No
READY	Converted and previewable; not yet live.	No
PUBLISHED	The one live report for the brand.	**Yes**
UNPUBLISHED	Was live (or was superseded); hidden. Can be republished.	No
FAILED	Conversion failed. Replace by uploading again.	No
DELETED (flag)	Soft-deleted by a manager. Removed from lists; assets purged later.	No

Rules: a brand has at most **one** PUBLISHED report at a time; publishing replaces it. Only READY or UNPUBLISHED reports can be published. A PUBLISHED report must be unpublished before it is deleted.

> **Assumption:** Publishing automatically unpublishes the previously live report. This avoids a surprising 'older week reappears' effect when the newest report is unpublished: in that case the client sees the empty state.

16. Versioning

- A **version** is one upload for a brand and reporting week. First upload is v1; each later upload for the same brand and week is v2, v3 and so on, assigned by the server in a transaction.
- Uploading never modifies an earlier version. The old version keeps its source file, slides and status.
- Only elevated users can see the version list. Clients only ever receive the single latest **published** report.
- Example: v1 of Week 42 is published and contains an error. Manager uploads v2, previews it, publishes it. v1 becomes UNPUBLISHED automatically; clients now see v2. v1 stays in history.
- Rolling back is a republish of an UNPUBLISHED version.

> **Assumption:** All versions are retained indefinitely, except those a manager deletes. Storage impact is modest (a deck is tens of MB); a retention policy can be added later without changing the model.

17. Data Model

Logical model. Names follow Datum SQL conventions: snake_case, `BIGINT UNSIGNED` auto-increment ids, `created_at` / `updated_at`, `brand_key VARCHAR(32)`.

weekly_reports

Column	Type (proposed)	Notes
id	BIGINT UNSIGNED PK	Report ID.
brand_key	VARCHAR(32) NOT NULL	Set from the signed header. Indexed. Never updated.
title	VARCHAR(200)	

Column	Type (proposed)	Notes
reporting_week	VARCHAR(40)	Label such as 'Week 42'.
version	INT UNSIGNED	Unique with (brand_key, reporting_week).
source_type	ENUM('pptx','ppt')	Future: 'pdf', 'canva'.
source_storage_key	VARCHAR(512)	Private. Never returned by any API.
source_sha256, source_size_bytes	CHAR(64), BIGINT	Integrity and diagnostics.
status	ENUM('PROCESSING','READY','PUBLISHED','UNPUBLISHED','FAILED')	Index (brand_key, status, published_at).
slide_count	INT UNSIGNED NULL	Set at READY.
error_code, error_detail	VARCHAR	Internal only; clients never see detail.
uploaded_by, published_by	VARCHAR(64)	User ids (the auth service uses UUID strings).
uploaded_at, published_at, created_at, updated_at	DATETIME	UTC.
deleted_at	DATETIME NULL	Soft delete.

weekly_report_slides

Column	Type	Notes
id	BIGINT UNSIGNED PK	
report_id	BIGINT UNSIGNED FK	Unique with slide_number.
slide_number	INT UNSIGNED	1-based.
image_storage_key, thumbnail_storage_key	VARCHAR(512)	Private keys.
width, height, created_at	INT, DATETIME	

weekly_report_events (audit)

Column	Type	Notes
id, created_at	BIGINT, DATETIME	
brand_key, report_id, user_id	VARCHAR / BIGINT	report_id nullable for denied attempts.
action	VARCHAR(40)	See section 24.

Column	Type	Notes
metadata	JSON	Version, reason, requested brand on denial, request id. No file contents.

Confirmed in repo: Datum has three data locations: shared Mongo (users, tenants), per-brand MySQL databases routed by `tenant-router`, and one central MySQL (`api_keys`, `dashboard_layouts`, which use `brand_key VARCHAR(32)` and `BIGINT UNSIGNED` ids). There is no migration tool: schema is hand-run `.sql` files.

Assumption: The three tables live in the **central MySQL** with a `brand_key` column, matching the `api_keys` pattern, rather than in each brand database. One schema, one migration, and the service needs no tenant routing. Trade-off: brand data is not physically separated, so the mandatory `brand_key` predicate (section 9) is the isolation mechanism. Listed as an open question.

Recommendation: Ship a numbered `.sql` migration in the new service (same style as `daily-insights-service/migrations`).

18. API Requirements

Routed through the gateway under a new prefix `/weekly-reports/`, like `/daily-insights/` and `/analytics/` (no version segment in Datum today). All calls carry `?brand_key=` from the client; the service uses only the signed header. Success returns plain JSON; errors return `{ "error": "message" }`, plus a stable `code` field for the UI.

Method and path	Access	Purpose
`GET /weekly-reports/latest`	view	Latest PUBLISHED report metadata for the brand (id, title, week, published_at, slide_count) or 404 `NO_PUBLISHED_REPORT`.
`GET /weekly-reports/:id`	view	Metadata. Clients: only the live published report of their brand. Elevated: any report of the current brand.
`GET /weekly-reports/:id/slides/:n`	view	Slide image bytes (watermarked). `?size=thumb` for thumbnails. Re-authorized on every request.
`GET /weekly-reports`	manage	List reports and versions for the current brand.
`POST /weekly-reports`	manage	Multipart upload: `file`, `title`, `reporting_week`. Returns 202 with the new report.
`POST /weekly-reports/:id/publish`	manage	READY or UNPUBLISHED to PUBLISHED; previous published becomes UNPUBLISHED (one transaction).
`POST /weekly-reports/:id/unpublish`	manage	PUBLISHED to UNPUBLISHED.
`DELETE /weekly-reports/:id`	manage	Soft delete; refused while PUBLISHED.

There is intentionally **no endpoint that returns the source file**, for any role, in V1.

Status codes

Code	When
400	Invalid input; a body or path brand that disagrees with the signed brand.
401	Missing or invalid authentication or gateway signature.
403	Missing permission, brand not allowed, or report belongs to another brand.
404	No such report, no published report, or a client asking for a report that is not live.
409	Illegal state change (for example publishing a PROCESSING report, deleting a PUBLISHED one).
413	File over the size limit.
415	Unsupported file type.
422	Corrupt file or too many slides.
503	Storage or database unavailable.

Confirmed in repo: Datum APIs use gateway prefixes, plain JSON, `{error: msg}` errors and zod validation. Rate limiting exists only at the gateway (per IP 100/min, per user 1000/min, per brand 5000/min, admin routes 50/min). A 20-slide deck with thumbnails stays well within the per-user limit; confirm the admin limit is not applied to this prefix.

Confirmed in repo: The gateway's nginx config has no `client_max_body_size`, so the 1 MB default would reject uploads with 413. The new location needs a body limit above the file limit plus longer proxy timeouts.

Assumption: A client requesting an older report of their own brand receives 404 (hidden). A request for a report of a different brand receives 403, matching the brief; see open question on uniform 404.

19. Storage Architecture

All report assets are stored in a **new dedicated private S3 bucket**, for example `datum-reports-prod`.

```
{brand_key}/{report_id}/source/presentation.pptx     (private, never served)
{brand_key}/{report_id}/slides/001.webp ...
{brand_key}/{report_id}/thumbnails/001.webp ...
```

- Block Public Access on; bucket policy denies any non-service principal; server-side encryption on; versioning optional.
- Service access through an IAM role with least privilege (Get/Put/Delete on this bucket only), preferably the EC2 instance profile rather than static keys.
- Slides are streamed through the service after the authorization checks. **No public or presigned URL is ever given to a client** in V1.
- Optional lifecycle rule: purge assets of soft-deleted reports after 30 days.

Decision: New dedicated private bucket, not the existing one.

Confirmed in repo: Datum already uses S3 (`@aws-sdk/client-s3`) and the analytics service has an upload route (`analytics/modules/uploads`). That route should **not** be reused: it uses memory storage with no size limit, relies on static `AWS_ACCESS_KEY` / `AWS_SECRET` variables, has no authorization of its own, and returns public-style object URLs. No presigner library is installed.

Recommendation: Stream slides through the service for V1. Reason: it keeps the per-request brand and permission check on the critical path and is simple. If load grows, put CloudFront in front with signed cookies; that is a later optimization.

20. Security Requirements

ID	Requirement
SEC-1	Backend authorization on every endpoint (authentication, permission, brand, report state). Frontend hiding is cosmetic.
SEC-2	Brand comes only from the gateway-signed header. Body, path and query values are never trusted to select a brand.

ID	Requirement
SEC-3	Every query filters by brand_key; report IDs are not guessable-safe, they are authorized.
SEC-4	The original file is stored privately and has no route; no download, export, share, print or open-original feature exists for clients.
SEC-5	No public object URLs. Slide responses use `Cache-Control: private`, `X-Content-Type-Options: nosniff`, `Content-Disposition: inline`.
SEC-6	Upload hardening: size cap, extension and signature check, zip-bomb guard, macro-enabled formats rejected, conversion in an isolated process with no network and a timeout, non-root container user.
SEC-7	The service fails closed when the gateway secret is missing.
SEC-8	Denied cross-brand and missing-permission attempts are written to the audit log.
SEC-9	Dynamic watermark on slides served to viewers (section 25).
SEC-10	Internal error detail (file paths, converter output) is never returned to clients.

What this does not guarantee

> **Limitation.** Datum cannot make content shown in a browser impossible to copy. A determined viewer can still take screenshots, photograph the screen or screen-record the viewer. The security requirement is therefore stated precisely: **the original presentation file is never exposed to the client, and Datum provides no download, export, sharing or original-file access functionality.** The watermark and the audit trail raise the cost and traceability of leaks; they do not prevent them.

21. Brand Isolation Test Matrix

Each row must be an automated test against the service (integration level) before release.

#	Scenario	Expected
T1	BBB user requests BBB latest report	200
T2	BBB user requests a PTS report ID	403 (and audit event)
T3	PTS user requests a BBB report ID	403
T4	User without `weekly_reports_panel` calls any view endpoint	403
T5	User with the permission but no membership for the brand in `brand_key`	403 (gateway)
T6	Elevated user, brand = BBB, requests a PTS report ID	403 (report is not in the current brand)
T7	Change `:id` to another brand's id	403; authorization is on the report's real brand
T8	Send `brand_key=PTS` in the body while the signed brand is BBB	400 or ignored; report stored under BBB
T9	Client calls `POST/DELETE/publish/unpublish`	403

#	Scenario	Expected
T10	Client requests an UNPUBLISHED, FAILED or PROCESSING report of own brand	404
T11	Client requests slide number out of range	404
T12	Request without gateway signature, or secret unset	401; service refuses to start without secret
T13	Search all responses and logs for `source_storage_key` or S3 URLs	None present
T14	Publish v2 while v1 live	Client sees only v2; v1 UNPUBLISHED

22. Performance Requirements

All numbers below are **proposed targets** for discussion, not existing requirements. Measure on a typical 20-slide deck on a 10 Mbps connection.

Metric	Proposed target
First slide visible after clicking View Report	p75 under 2 seconds
Next slide when prefetched	under 300 ms
Average slide image size	under 300 KB (WebP, 1920 px)
Thumbnail size	under 25 KB each
Metadata endpoint	p95 under 300 ms
Conversion of a 20-slide deck	p90 under 2 minutes
Viewer download before slide 1	slide 1 and the thumbnail rail only; never the whole deck

- Lazy-load thumbnails; prefetch the next two and previous one slide; cap the in-memory window of decoded slides.
- Per-viewer watermarking happens when a slide is served; cache the watermarked result briefly per user and slide.
- Conversion competes for CPU and memory with other containers on the EC2 host (Kafka runs there with a memory limit). Size the worker container accordingly and cap concurrency.

23. Error Handling

Situation	Behavior	User message (internal / client)
Unsupported file type	415, nothing stored	'Only .pptx or .ppt files are supported.'
File too large	413, nothing stored	'File is larger than 50 MB.'
Corrupt or unreadable PPTX	422, report FAILED if already created	'This file could not be read. Re-export and try again.'

Situation	Behavior	User message (internal / client)
Conversion failure	FAILED, error code stored, audit event	Internal: reason and Replace button. Client: never visible.
Processing timeout	FAILED `PROCESSING_TIMEOUT`	Internal: 'Conversion took too long.'
No published report	404 `NO_PUBLISHED_REPORT`	Client: 'No report has been published for BBB yet.'
Unauthorized user	401 / 403, frontend redirects to the existing Unauthorized page	Generic; no detail about what exists.
Unauthorized brand	403, audit event	Generic.
Missing slide asset	404 for that slide; viewer shows retry for that slide only	'This slide could not be loaded.'
Storage failure	503, retried by the client; logged	'Report is temporarily unavailable.'

Confirmed in repo: Existing error patterns to reuse: `{error: message}` bodies, the `Unauthorized.jsx` and `MaintenanceScreen.jsx` screens, 502/503/504 treated as maintenance in `api.js`, and `react-toastify` toasts.

24. Audit Logging

Events recorded in `weekly_report_events`:

Action	When
REPORT_UPLOADED	Upload accepted (new report row).
REPORT_VERSION_CREATED	Upload is v2 or later for the week.
REPORT_PROCESSED / REPORT_PROCESSING_FAILED	Worker finishes.
REPORT_PUBLISHED / REPORT_UNPUBLISHED	State change, including automatic unpublish of the previous report.
REPORT_DELETED	Soft delete.
ACCESS_DENIED	Cross-brand or permission-denied attempt on a report.

Fields: user id, brand key, report id, action, timestamp, and metadata (version, reason, requested brand on denial, request id). Client viewing events (`REPORT_VIEWED`) are a V1.1 item.

Confirmed in repo: Datum has no general audit or activity log. The closest items are the health-monitor notification log and fields on `api_keys`. Current codebase does not appear to provide a reusable audit mechanism; implementation will require introducing the `weekly_report_events` table (kept local to this service rather than a platform-wide audit system).

25. Watermarking

What it does

- Overlays a low-opacity diagonal text on every slide shown to a viewer: *CONFIDENTIAL, brand, viewer email, date*.
- Discourages casual sharing of screenshots.
- Lets the team trace a leaked screenshot back to a user and a date.

What it does not do

- It does **not** prevent screenshots, photos or screen recording.
- It does not protect against cropping a determined leaker's view of the slide.
- It is not an access-control mechanism.

Decision: Watermark is included in V1.

Recommendation: Burn the watermark into the image on the server when the slide is served, instead of a CSS overlay in the browser. Reason: a CSS overlay can be removed in the browser's developer tools in seconds. Cost: slightly more CPU per slide request, mitigated by a short per-user cache and by the fact that decks are small.

Assumption: Text content is brand name, viewer email and date; internal preview shows the same watermark so managers see what clients see.

26. Canva (Future Extension)

V1 does **not** depend on Canva. Nothing in the repository integrates with Canva today, and a reliable import needs a supported export route.

Desired future flow:

Canva URL → Datum import (authorized Canva connection) → export to PPTX or PDF → Datum private storage → same conversion pipeline → Datum viewer

- The client is never redirected to Canva as the viewer; that would bypass Datum's access control and watermark.
- The data model already has `source_type` so new sources fit without redesign.
- Until then, the Product Team exports the Canva design to PPTX and uploads it.

Open question: Before V2, confirm which Canva capability and plan can export a design programmatically, and who owns the Canva authorization. This PRD does not assume any specific Canva API behavior.

27. V1 vs V2 Scope

Capability	V1	V1.1	V2+
Upload PPT/PPTX, convert, preview, publish, unpublish	Yes		
Brand scoping and backend authorization	Yes		

Capability	V1	V1.1	V2+
Versioning with internal history	Yes		
Delete (soft)	Yes		
Viewer: navigation, thumbnails, fullscreen, keyboard	Yes		
Dynamic watermark (server-side)	Yes		
Audit events for management actions and denied access	Yes		
Client view tracking (REPORT_VIEWED, internal view report)		Yes	
Zoom		Yes	
PDF upload (better fidelity)		Consider	
Virus scanning of uploads		Consider	
Canva URL import			Yes
Client access to history, email notification on publish			Consider

28. Acceptance Criteria

Upload

ID	Criterion (testable)
AC-U1	A user with an elevated role can upload a valid PPTX and receives 202 with a report in PROCESSING.
AC-U2	The stored report's brand equals the brand in the gateway-signed header; the form has no brand field.
AC-U3	A request whose body or path names a different brand is rejected (400) and nothing is stored.
AC-U4	A non-PPT file, a file renamed to .pptx, an oversize file and a macro-enabled file are each rejected with 415/413/422 and nothing is stored.
AC-U5	Status moves PROCESSING to READY with a correct slide count; the UI shows progress and updates without manual refresh.
AC-U6	A corrupt file or timeout ends in FAILED with a reason visible to internal users and no visibility to clients.

Permissions

ID	Criterion
AC-P1	Without `weekly_reports_panel` the menu item is absent, the route redirects, and every view endpoint returns 403.
AC-P2	A client with the view permission calling any manage endpoint receives 403, including by crafting the request directly.
AC-P3	Elevated roles can use view and manage endpoints for the current brand.
AC-P4	The service rejects requests with a missing or wrong gateway signature and does not start without the shared secret.

Client

ID	Criterion
AC-C1	The client sees exactly the latest published report of the active brand, or an empty state.
AC-C2	The client can move by buttons, thumbnails, keyboard and swipe, see 'n / total', and enter and leave fullscreen.
AC-C3	The UI and API expose no download, share, export, print, open-original or Canva-source function; there is no route to the source file.
AC-C4	No response, page source or log visible to the client contains a storage key or bucket URL.

ID	Criterion
AC-C5	Client cannot upload, delete, publish, unpublish or replace (UI absent, API 403).
AC-C6	Every slide shows the viewer's watermark (brand, email, date).
AC-C7	Slide 1 appears without downloading the other slides.

Brand isolation

ID	Criterion
AC-B1	All scenarios T1 to T14 in section 21 pass.
AC-B2	Switching the active brand changes the panel content to that brand's report only.

Versioning

ID	Criterion
AC-V1	Uploading again for the same brand and week creates v2 and leaves v1's source, slides and history intact.
AC-V2	Only the PUBLISHED report is returned to clients; READY, FAILED, UNPUBLISHED and PROCESSING are not.
AC-V3	Publishing v2 makes v2 the client-visible report and moves v1 to UNPUBLISHED in a single transaction.
AC-V4	A PUBLISHED report cannot be deleted; an unpublished one can, and disappears from lists.

Audit

ID	Criterion
AC-A1	Upload, publish, unpublish, delete, processing result and denied access each produce one audit row with user, brand, report and time.

29. Assumptions and Open Questions

Assumptions (made without explicit confirmation)

ID	Assumption	If wrong
A1	Maximum file size 50 MB; maximum 100 slides.	Change limits; adjust gateway body limit.
A2	All versions retained indefinitely; soft-deleted assets purged after 30 days.	Add retention job.
A3	Internal users (not clients) can see version history.	Hide history.
A4	Tables live in the central MySQL with a `brand_key` column.	Move to per-brand databases (needs tenant routing).
A5	Publishing auto-unpublishes the previously published report; one live report per brand.	Allow several live reports; define 'latest' by published_at.

ID	Assumption	If wrong
A6	Conversion with LibreOffice headless is good enough for the decks the team produces.	Add PDF upload earlier, or a different renderer.
A7	The central MySQL supports `SKIP LOCKED` (MySQL 8).	Use a different claim mechanism.
A8	Watermark text is brand, viewer email and date.	Adjust text.

Decisions already made with the product owner

Question	Answer
How should permissions work?	Authors/admins have everything. Clients get one view-only permission.
Where does the backend live?	New `weekly-reports-service`.
Which storage?	New dedicated private bucket.
V1 scope extras	Watermark and delete are in V1; view tracking is deferred to V1.1.

Open questions to resolve before implementation

ID	Question	Why it matters
Q1	Central MySQL or per-brand databases for the new tables? Which MySQL version?	Isolation model, job claiming.
Q2	Confirm 50 MB / 100 slide limits and indefinite retention.	Gateway config, storage cost.
Q3	Should cross-brand report access return 403 (as specified) or a uniform 404 so existence is not revealed?	Information leakage versus clearer errors.
Q4	Should internal users be able to download the original file from Datum? V1 says no for everyone.	Product team may still want a copy.
Q5	Allow PDF upload in V1 for better fidelity?	Avoids LibreOffice rendering differences.
Q6	Do brand fonts used in decks need to be installed in the worker image? Which?	Slide fidelity.
Q7	Is there EC2 capacity for LibreOffice (about 1 to 2 GB RAM during conversion) beside Kafka and the other services?	Host sizing.
Q8	Is client view tracking wanted soon (V1.1), and who should see it?	Schema and privacy notice.

30. Implementation Considerations

Technical guidance for engineering. Items marked *Existing* were found in the repository; *New* items do not exist today.

Area	Change	Status
api-gateway: `src/constants/permissions.js`	Add `weekly_reports_panel` to `ALL_PERMISSIONS`.	Existing file, small change

Area	Change	Status
api-gateway: `gateway/nginx.conf`	New `/weekly-reports/` upstream and location; raise `client_max_body_size` and timeouts for upload only.	Existing file, new block
docker-compose and deploy workflows	Add the service, env, volume for temp conversion files, memory limit; join the gateway network.	Existing files
weekly-reports-service	Express service modeled on `daily-insights-service` (auth middleware with signature check that fails closed, zod validation, SQL migrations). Includes API, worker and storage client.	New
Worker image	LibreOffice (Impress), fonts, poppler or an equivalent PDF rasterizer, an image library for WebP and watermark.	New (none exists)
AWS	New private bucket, IAM role, optional lifecycle rule. Do not hardcode keys.	New
client: `Sidebar.jsx`, `App.jsx`	Add nav item, entry in `TAB_ROUTE_MAP`, `canAccessWeeklyReports`, `accessibleTabs`, redirect guard and render case. Update the second tab list and `MobileNav.jsx`.	Existing, multiple places
client: `AccessControlCard.jsx`	Add the permission to `PERMISSION_OPTIONS` and the Dashboard group.	Existing
client: new route container and components	`WeeklyReportsRouteContainer`, panel, uploader, report card, version table, viewer.	New
client: `lib/api.js`	Add helpers; verify the POST helper can send `FormData` without forcing a JSON content type; add blob fetch for slides.	Existing file, verify

Notes

- Use the existing `x-permissions` / role headers for checks, but add the new service-level `manage` check as an elevated-role test.
- Run the conversion as a separate process from the API inside the service so a hung LibreOffice cannot block requests.
- Write unit tests for state transitions and brand predicates, and integration tests for the section 21 matrix, using the repo's existing Jest setup.
- Rollout: ship behind the permission (nobody has it until granted); pilot with one brand; then grant to client users.

> **Confirmed in repo:** Deployment is Docker Compose on EC2 via GitHub Actions SSH (`DEPLOY_EC2.md`, `.github/workflows/deploy-ec2-*.yml`). The frontend ships from `client/dashboard` (Vite build, `vercel.json` present). Nothing in this PRD was implemented.

31. Future Enhancements

- Canva import (section 26) and PDF upload.
- Client view tracking and an internal 'who viewed' report.
- Client-visible history of past weeks (if the business wants it).

- Email or in-app notification when a new report is published.
- Zoom and slide-level comments.
- CloudFront signed delivery for scale.
- Virus scanning for uploads.